

Rapport lab07 - Boutique de barbier

Général

- Auteurs: Fabien Léger & Aymeric Bonny
- Course: PCO, HEIG-VD
- Date: 25.01.2026

Problématique

Il nous était demandé dans ce laboratoire de mettre en place la boutique d'un barbier. On aurait le barbier et divers clients qui entrent dans le magasin. Si le barbier dort, car il n'a pas de clients, le premier client doit le réveiller. Les clients suivants doivent se mettre en attente sur les chaises prévues à cet effet.

Dans le cas où il n'y a plus de places, les clients partiront faire une jolie promenade en attendant de pouvoir se faire une superbe coupe plus tard dans la journée.

Après s'être coupé les cheveux, un client attendra qu'ils repoussent avant de revenir.

Lorsque le magasin ferme, le barbier coupe tout de même les cheveux des clients déjà présents.

Implémentation

Nous avons décidé de partir sur un moniteur de Hoare car nous trouvons son utilisation plus simple, notamment le fait qu'une queue FIFO soit déjà mise en place.

Un problème est survenu toutefois étant que lorsqu'une animation est lancée, cela fait `monitorOut()` puis `moniteurIn()` après la fonction. Cela veut dire que nous perdons cette FIFO et que l'ordre d'arrivée dans le magasin ne décide pas nécessairement de comment se placent les clients.

Barber & Client

Il n'y a pas eu beaucoup à faire pour les `run()` des deux classes. On a suivi les machines d'états de la donnée. Les boucles tournent tant que la boutique n'est pas fermée. Le barbier doit également continuer tant qu'il y a des clients dans le magasin.

PcoSalon

Voici une description des différents attributs de PcoSalon :

Nom	Type	Description
barberSleeping	Condition	Condition sur laquelle le barbier attend jusqu'à être réveillé
clientWaiting	Condition	Condition sur laquelle les clients sur les chaises d'attentes attendent
barberWaitsAtChair	Condition	Condition sur laquelle le barbier attend jusqu'à ce que le client soit sur la chaise de travail
clientBeautifying	Condition	Condition sur laquelle le client attend jusqu'à la fin de sa magnificence
_nb_sieges	const unsigned int	Nombre total de chaises d'attente
seats	bool*	Tableau des chaises libres
nbClientsWaiting	unsigned int	Nombre de clients sur les chaises d'attente
isBarberSleeping	bool	True si le barbier dort, false sinon
isClientReady	bool	True si le premier client a réveillé le barbier et n'a pas besoin d'être choisi, false sinon
isClientOnChair	bool	True si le client est sur la chaise de travail, false sinon
haircutDone	bool	True si le barbier vient de finir la coupe du client, false sinon
isSalonInService	bool	True si la boutique du barbier est ouverte, false sinon

Pour ce qui est des variables de condition et des quatre premiers booléens, ils servent à synchroniser le barbier et les clients ainsi que leurs animations respectives.

Les variables `_nb_sieges`, `seats`, `nbClientsWaiting` sont une description des chaises d'attente. En y réfléchissant, il aurait été possible de faire un array, mais les variables se sont rajoutées au cours du temps.

Le booléen `isSalonInService` sert à faire ressortir les clients réentrant et à donner l'information à l'extérieur de la classe que la boutique est fermée. Cela donnera fin au programme avec le temps.

Tests

Il n'y a pas eu de tests codés, mais nous avons lancé et arrêté le code plusieurs fois à différents moments et cela semble marcher selon la donnée du laboratoire.

Conclusion

Ce laboratoire a permis une liberté sur le choix de l'outil utilisé que nous avons tout particulièrement apprécié. Les animations nous ont posés quelques problèmes à cause des effets que cela posait pour le moniteur. En lâchant notre propriété du moniteur, on laisse la possibilité à d'autres threads de la prendre. On pourrait imaginer mettre des sémaphores supplémentaires afin d'en faire réellement une FIFO et d'en améliorer le fonctionnement.